can see this in figure 17.1, where the “leap” the algorithm has to make to get itself out of a local minimum is pretty large, compared to the other updates: if such a large delta is even allowed (depending on how the algorithm is configured through its hyper-parameters), it will likely take a lot of random attempts before it generates a step both in the right direction and sufficiently far away.

17.1.1 Sometimes You Need to Climb Up to Get to the Bottom

An example of an even more troubling configuration for the local optimization algorithm is shown in figure 17.3: the gap between the current minimum where the randomized algorithm is stuck and the next point in the cost function’s landscape with a lower value (the closest improvement) is way too far away for to get there in a single step. What we would need, is for the algorithm to be able to say: fine, doesn’t matter if I’m in a minimum, I’ll climb over this hill and see if the next valley (or the one after the next) is deeper.

Of course, there are many ways to do that. For instance, we could store somewhere the best solution we have found, and keep exploring, moving past a local minimum. Another option could be deciding that sometimes a step uphill is fine: we can plan it systematically, every few steps, or probabilistically, accepting a step uphill with a certain probability.

Simulated annealing uses the latter approach; while in its original formulation it doesn’t keep track of the best